

Teamcenter engineering process management - How to configure a Teamcenter RAC headless Eclipse project

The purpose of a RAC headless application is to take advantage of the java RAC interface and allow developers to write custom RAC applications without using the traditional Eclipse UI interface, hence the term headless. This same objective can also be achieved with using Teamcenter SOA and in most cases would be the preferred way in customizing Teamcenter. The main difference between the RAC headless and SOA approach is the data model used; RAC uses the Teamcenter component data model and SOA uses the ModelObject model.

The goal of this document is to demonstrate how to create and debug a Teamcenter headless Eclipse project.

Before we start with the steps to create an Eclipse headless project, let's establish the prerequisites used to create this document.

- Teamcenter 8.1.x installed with 2 tier RAC
- Java 1.6.x SDK
- Eclipse 3.5.x with PDE SDK

Please see the *Rich Client Customization Programmer's Guide*, *Getting started* section for more information in how to setup Eclipse for Teamcenter development.

Configuring eclipse.ini file:

To insure Eclipse is using the correct JRE it is recommend the eclipse.ini file be updated to explicitly specify which JRE to use.

To do this, open the eclipse.ini (found in the eclipse folder) in your favorite text editor.

Add the following two lines to the file:

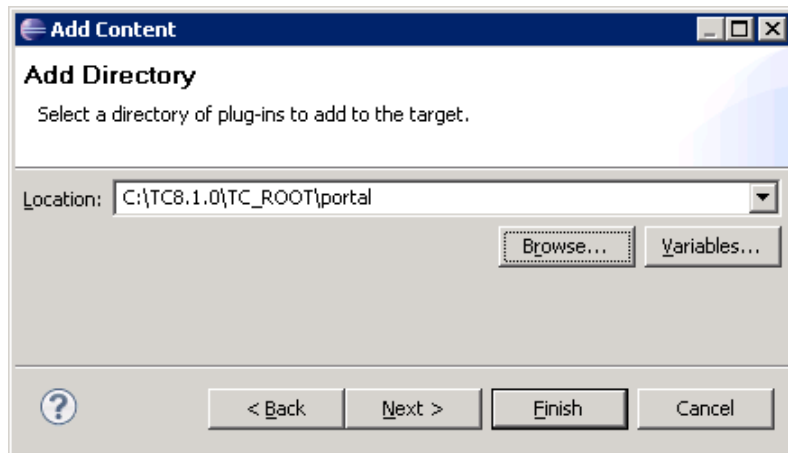
```
-vm  
C:\java\jdk1.6.0_17\jre\bin
```

The above two lines are usually placed before the *-vmargs line* in the eclipse.ini file and be sure to update the java 1.6.x path accordingly.

Save the file and start Eclipse.

Creating a new Eclipse workspace and configuring the preferences:

1. From the menu, select, **File => Switch Workspace => Other**.
2. Enter the appropriate workspace path and call the workspace:
HeadlessWorkspace
3. From the menu, select, **Windo => Preferences**; expand the Java option, select **Installed JREs**.
4. If you do not see your java 1.6.x jre listed add it. Make it is also the default JRE to be used.
5. Expand **Plug-in Development**, and select, **Target Platform**.
6. Click, **Add**, and select option, **Nothing**: Start with an empty target definition.
7. Click, **Next**.
8. In the Location tab for the Name, enter: **Headless**.
9. Click **Add => Directory**, and enter the directory to the Teamcenter RAC portal folder.



10. Click **Finish**.
11. At this point the portal\plugin folder is read.
12. Click **Finish**.
13. Check Headless for Active Target Definition.
14. Click **Apply** and **Ok**.

Creating an Eclipse Project:

1. Close the welcome screen and the default perspective should be "Java."
2. In the **Package Explore** view, right click, **New => Project**.
3. From the project options, select "**Plug-in Project**" and click **Next**.
4. For the project name, enter: **com.gtac.headless**
5. For Target Platform, select, **OSGi framework, Equinox**.

New Plug-in Project

Plug-in Project
Create a new plug-in project

Project name:

☒ Use default location

Location:

Project Settings

☒ Create a Java project

Source folder:

Output folder:

Target Platform
This plug-in is targeted to run with:

☐ Eclipse version:

☒ an OSGi framework:

Working sets

☐ Add project to working sets

Working sets:

6. Click **Next**.

New Plug-in Project

Content
Enter the data required to generate the plug-in.

Properties

ID:

Version:

Name:

Provider:

Execution Environment:

Options

☒ Generate an activator, a Java class that controls the plug-in's life cycle
Activator:

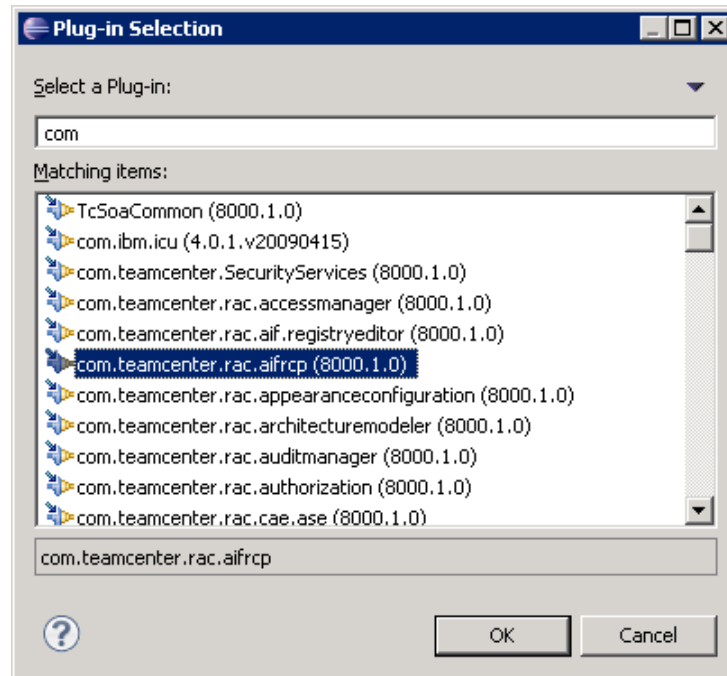
☐ This plug-in will make contributions to the UI

☐ Enable API Analysis

7. By default you should have the above screen.
8. Click **Next**.
9. Deselect: **Create a plug-in using one of the templates**.
10. Click **Finish**.
11. Your project should be automatically created and switched to Plug-in Development perspective.

Configuring the Manifest.mf file:

1. Open the **Manifest.mf** file (If it is not open already)
2. In the bottom tabs for this file, select, **Dependencies**.
3. In the **Required Plug-in** area, click **Add...**
4. Enter com to show matching items.



5. Select the following items:

com.teamcenter.rac.aifrcp
com.teamcenter.rac.kernel
com.teamcenter.rac.util

6. Click **Ok**.
7. Click **Add again**, and add item:

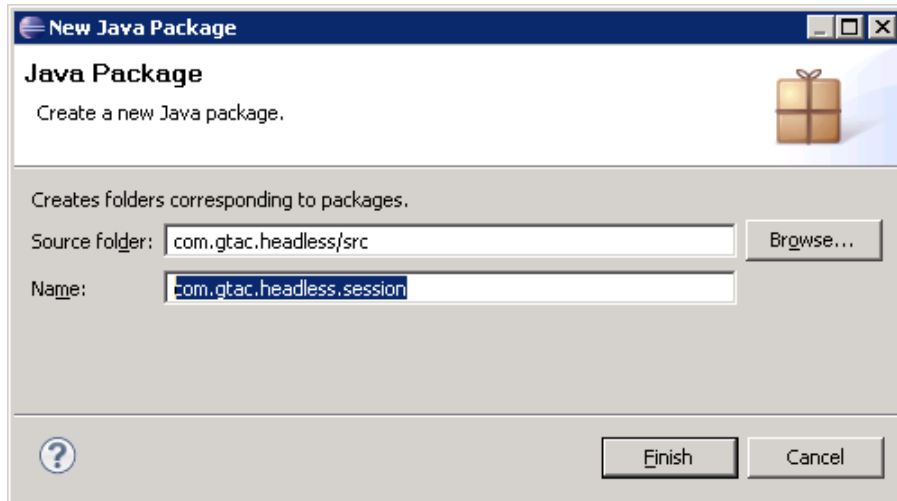
TcSoaCoreRac

8. Save the Manifest file.
9. The project **Manifest.mf** file should look something like this:

```
Manifest-Version: 1.0
Bundle-ManifestVersion: 2
Bundle-Name: Headless
Bundle-SymbolicName: com.gtac.headless
Bundle-Version: 1.0.0.qualifier
Bundle-Activator: com.gtac.headless.Activator
Bundle-Vendor: GTAC
Bundle-RequiredExecutionEnvironment: JavaSE-1.6
Import-Package: org.osgi.framework;version="1.3.0"
Bundle-ActivationPolicy: lazy
Require-Bundle: com.teamcenter.rac.aifrcp;bundle-version="8000.1.0",
com.teamcenter.rac.kernel;bundle-version="8000.1.0",
com.teamcenter.rac.util;bundle-version="8000.1.0",
TcSoaCoreRac;bundle-version="8000.1.0"
```

Adding Java source:

1. Create a new package called: **com.gtac.headless.session**



2. In this new package create a new java class called: **Session**.
3. Click **Finish**.
4. Update the source for **Session.java** with the following code:

```
package com.gtac.headless.session;
```

```
import com.teamcenter.rac.aifrcp.AifrcpPlugin;  
import com.teamcenter.rac.kernel.TCSession;  
import com.teamcenter.rac.kernel.TcServiceGatewayCorbaConnection;  
import com.teamcenter.rac.kernel.TcServiceGatewayWebServiceConnection;  
import com.teamcenter.rac.services.ISessionService;  
import com.teamcenter.rac.util.Registry;
```

```
public class Session  
{  
    private TCSession tcSession = null;  
    private String user = null;  
    private String password = null;  
  
    private String serverHost = "localhost";  
    private String serverName = "TcData";  
    private String serverMarker = "TcServer1";  
  
    private String URL = "http://localhost:7001/tc";  
    private int port = 1572;  
  
    public Session(String user, String password)  
    {  
        this.user = user;  
        this.password = password;  
    }  
  
    public boolean hasTCSession()  
    {  
        return (tcSession != null) ? true : false;  
    }  
  
    public TCSession getTCSession()
```

```

    {
        return tcSession;
    }

    public void setPort(int port)
    {
        this.port = port;
    }

    public void setHost(String host)
    {
        this.serverHost = host;
    }

    public void setServerNameAndMarker(String name, String marker)
    {
        this.serverName = name;
        this.serverMarker = marker;
    }

    public void setURL(String url)
    {
        this.URL = url;
    }

    public void login()
    {
        System.setProperty("java.awt.headless", "true");
        Registry registry = Registry.getRegistry("site_specific");
        String transportInUse = registry.getString("portalCommunicationTransport");
        System.out.println(this.getClass().getName() + ".login(): Transport protocol: " +
times    transportInUse);

        try
        {
            ISessionService iss = AifrcpPlugin.getSessionService();

            if(iss == null) // the SessionService may still be comming up...try a few
times
            {
                int numtry = 5;

                do
                {
                    Thread.sleep(50);
                    iss = AifrcpPlugin.getSessionService();
                    numtry--;
                } while(iss == null && numtry > 0);

                if(iss == null)
                {
                    System.out.println(this.getClass().getName() + ".login():
No SessionService");

                    return;
                }
            }
        }
    }

```

```

        tcSession =
(TCSession)jss.getSession("com.teamcenter.rac.kernel.TCSession");

        if(transportInUse.compareTo("iiop") == 0)
        {
            TcServiceGatewayCorbaConnection serverConnection = new
TcServiceGatewayCorbaConnection(serverHost, serverName);
            serverConnection.setMarkerServerName(serverMarker);
            serverConnection.setPort(port);

            if(serverConnection.connect())
            {
                tcSession.login(serverConnection, user, password, "",
"");
            }
        }
        else if(transportInUse.compareTo("http") == 0)
        {
            TcServiceGatewayWebServiceConnection connection = new
TcServiceGatewayWebServiceConnection(URL, serverName);

            if(connection.connect())
            {
                tcSession.login(connection, user, password, "", "");
            }
        }
    }
    catch(Exception ex)
    {
        System.out.println(this.getClass().getName() + ".login(): Failed");
        ex.printStackTrace();
    }
}

public void logout()
{
    try
    {
        if(tcSession != null)
        {
            tcSession.logout();
        }
    }
    catch(Exception e)
    {
        System.out.println(this.getClass().getName() + ".logout(): Failed");
        e.printStackTrace();
    }
}
}

```

5. For your environment you may need to update variables, **serverHost**, **serverName**, **serverMarker**, and URL.
6. Create a new package called: **com.gtac.headless.tasks**.

7. In this new package create a class called: **CreateItem**.
8. Update the source for **CreateItem.java** with the following code:

```
package com.gtac.headless.tasks;

import com.teamcenter.rac.kernel.TCComponentItem;
import com.teamcenter.rac.kernel.TCComponentItemRevision;
import com.teamcenter.rac.kernel.TCComponentUser;
import com.teamcenter.rac.kernel.TCException;
import com.teamcenter.rac.kernel.TCSession;
import com.teamcenter.services.rac.core.DataManagementService;

public class CreateItem
{
    private TCSession tcSession = null;
    private DataManagementService dservice = null;

    private TCComponentItem item = null;
    private TCComponentItemRevision itemRev = null;

    public CreateItem(TCSession session)
    {
        this.tcSession = session;
        this.dservice = DataManagementService.getService(session);
    }

    public void createItem(String clientID)
    {
        try
        {
            DataManagementService.ItemProperties[] properties = new
DataManagementService.ItemProperties[1];
            DataManagementService.ItemProperties property = new
DataManagementService.ItemProperties();
            TCComponentUser user = tcSession.getUser();

            property.clientId = clientID;
            property.description = "Export Item";
            property.extendedAttributes = new
DataManagementService.ExtendedAttributes[0];
            property.itemId = "";
            property.name = "";
            property.revId = "";
            property.type = "";
            property.uom = "";

            properties[0] = property;

            DataManagementService.CreateItemsResponse itemResp =
dservice.createItems(properties, user.getHomeFolder(), "");

            if(itemResp.output.length > 0)
            {
                item = itemResp.output[0].item;
                itemRev = itemResp.output[0].itemRev;
            }
        }
    }
}
```

```

        }
    }
    catch(TCException e)
    {
        e.printStackTrace();
    }
}
}

```

9. Update the source for **Activator.java** with the following code:

```
package com.gtac.headless;
```

```
import org.osgi.framework.BundleActivator;
import org.osgi.framework.BundleContext;
import com.gtac.headless.session.Session;
import com.gtac.headless.tasks.CreateItem;
```

```
public class Activator implements BundleActivator
{
    Session session = null;

    /*
     * (non-Javadoc)
     * @see org.osgi.framework.BundleActivator#start(org.osgi.framework.BundleContext)
     */
    public void start(BundleContext context) throws Exception
    {
        session = new Session("infodba", "infodba");
        session.login();

        if(session.hasTCSession())
        {
            CreateItem item = new CreateItem(session.getTCSession());

            item.createItem("my RAC Item");
        }
    }

    /*
     * (non-Javadoc)
     * @see org.osgi.framework.BundleActivator#stop(org.osgi.framework.BundleContext)
     */
    public void stop(BundleContext context) throws Exception
    {
        if(session != null)
        {
            session.logout();
        }
    }
}

```

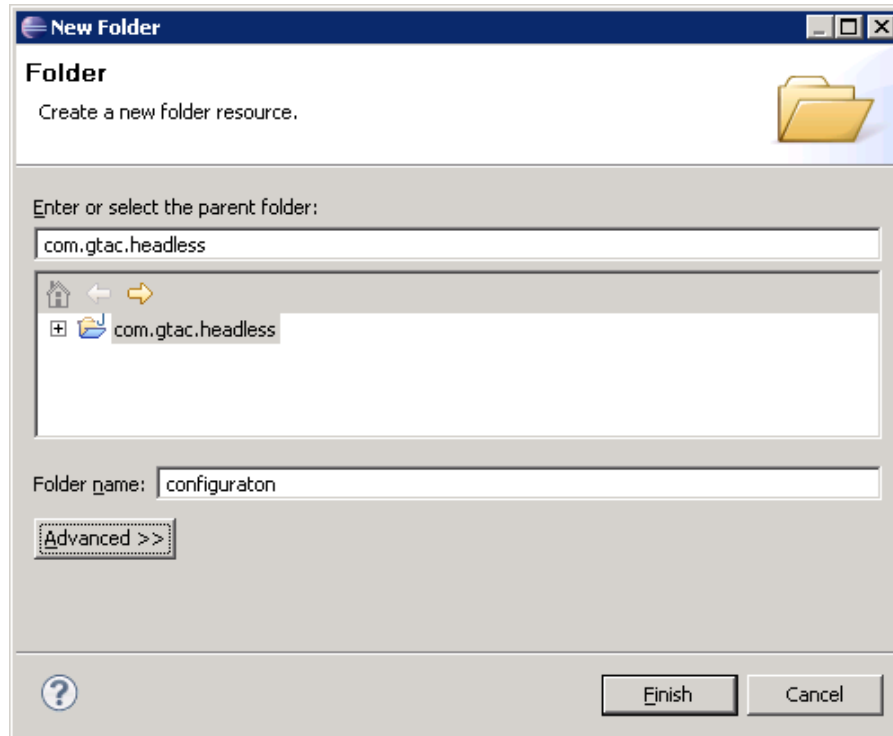
10. If needed, update the Session User and Password to reflect your installation.

Setting a debug Break point:

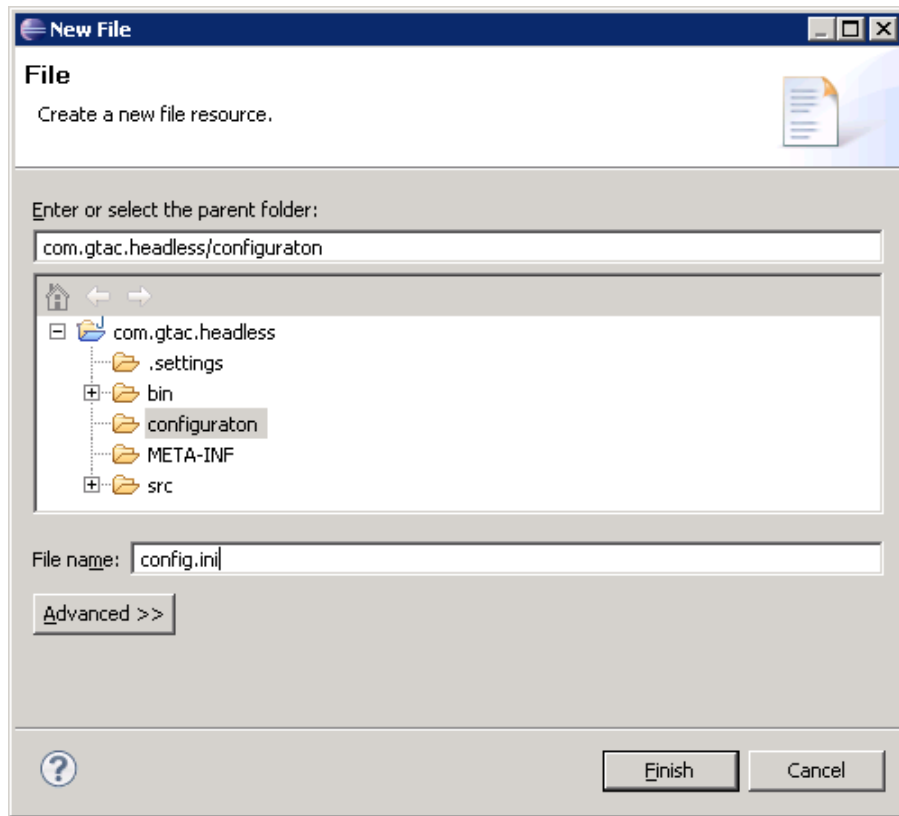
1. While viewing the **Activator.java** file, on the far left of line:
`session = new Session("infodba", "infodba");`
In the gray vertical bar, right click, **Toggle Breakpoint**.
2. A blue dot should appear in the grey bar representing a breakpoint.

Adding the config.ini file:

1. In the Package Explorer, select the root node (**com.gtac.headless**).
2. Right click, **New => Folder**.



3. For Folder name, enter: **configuration**.
4. Click **Finish**.
5. On the new configuration folder, right click, **New => File**.

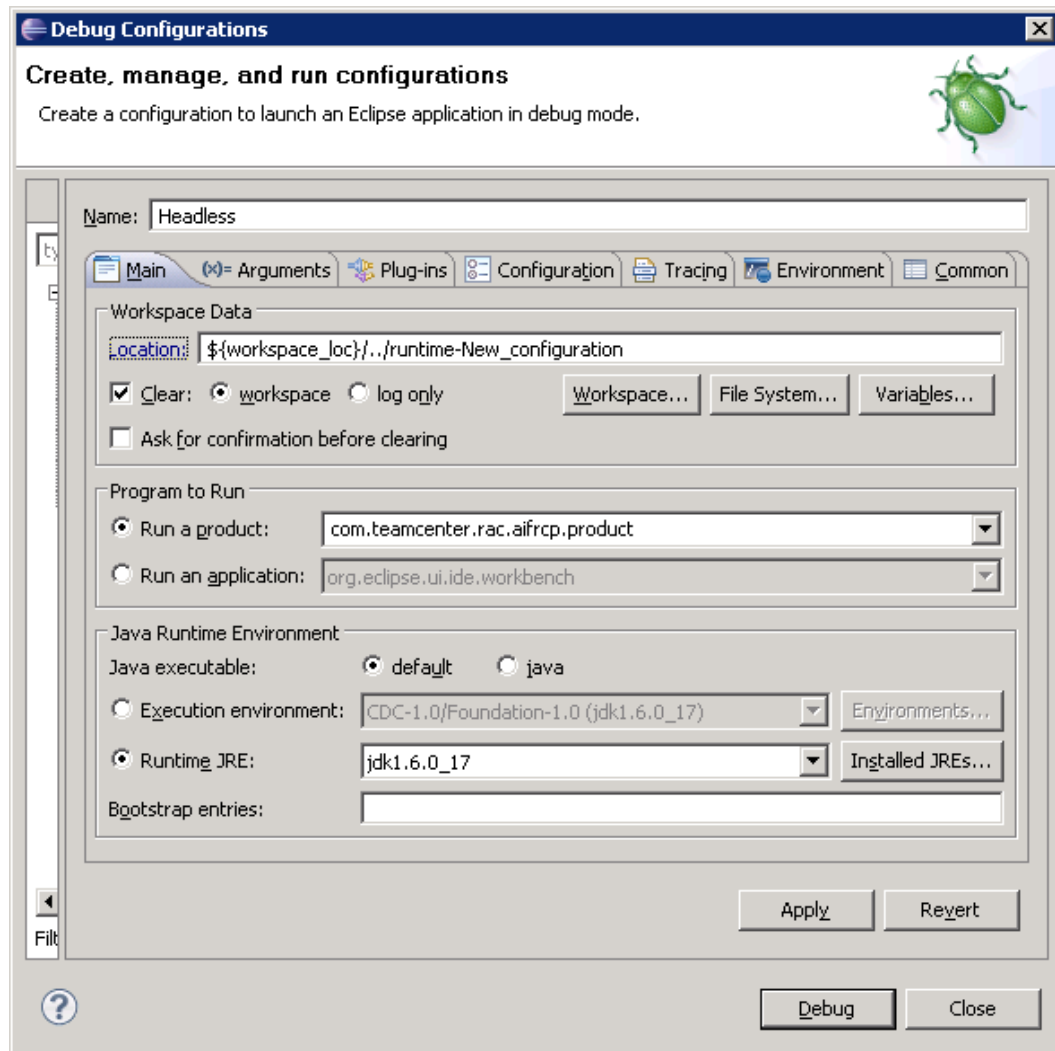


6. For File name, enter: **config.ini**.
7. Click **Finish**.
8. This should start your default Text editor (i.e. Notepad).
9. Enter the following for the file content:

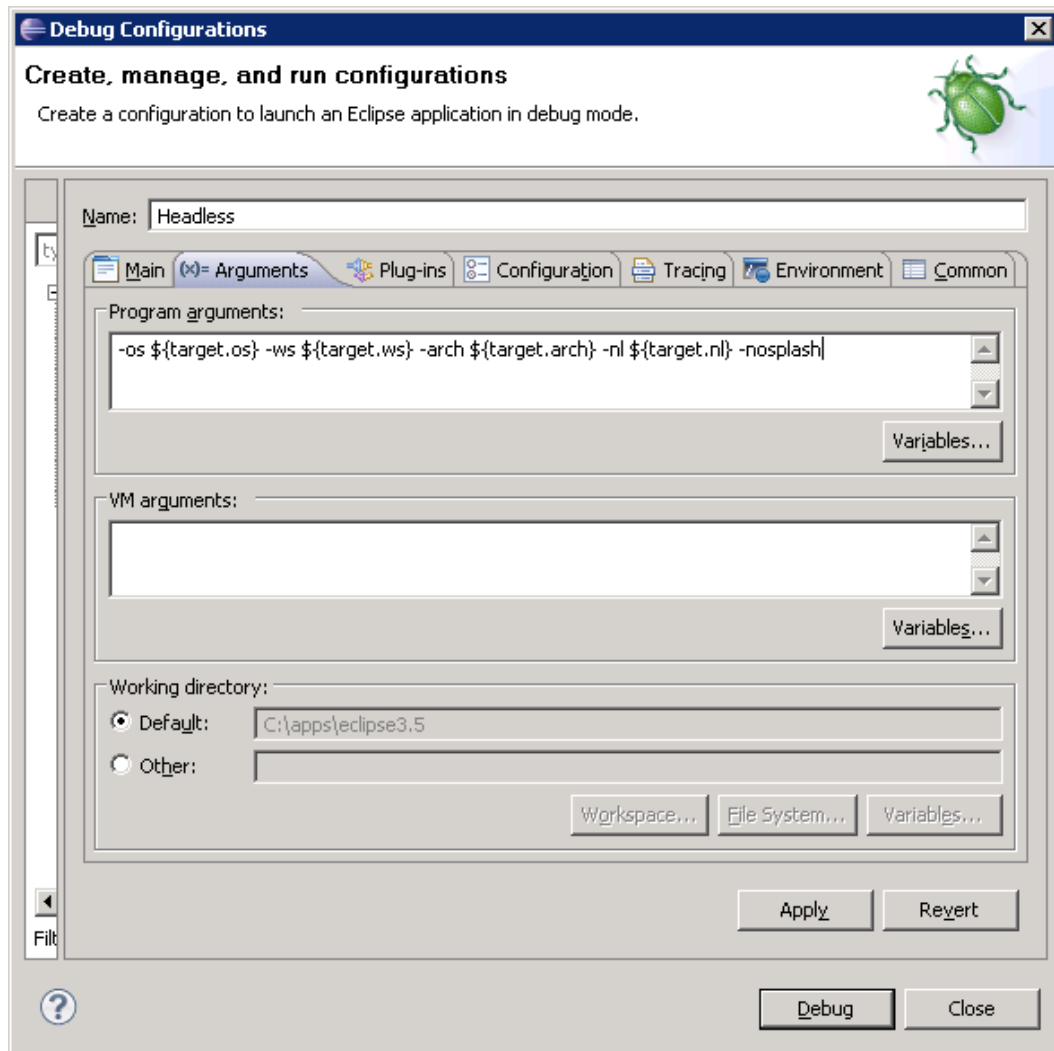
```
org.osgi.framework.bootdelegation=*
org.osgi.bundles=org.eclipse.equinox.common@2:start,org.eclipse.update.configurator@3:st
art,org.eclipse.core.runtime@3:start,com.gtac.headless@5:start
org.osgi.bundles.defaultStartLevel=5
org.osgi.noShutdown=false
eclipse.ignoreApp=true
```
10. **Note:** all the package names that follow `org.osgi.bundles=` are to be on ONE line. No LINE BREAKS or extra WHITESPACE.
11. Save the file.

Configuring the debug environment:

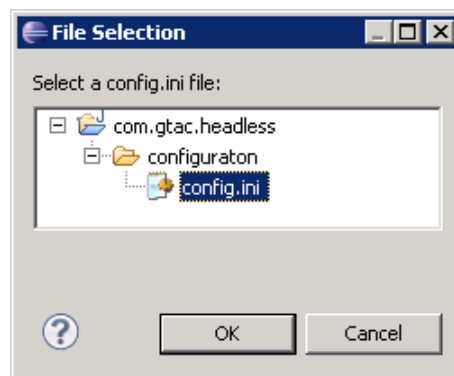
1. From the menu, select, **Run => Debug Configuration...**
2. Select Eclipse application and click the **New Configuration** icon.
3. On the Main tab for **Name**, enter: **Headless**.

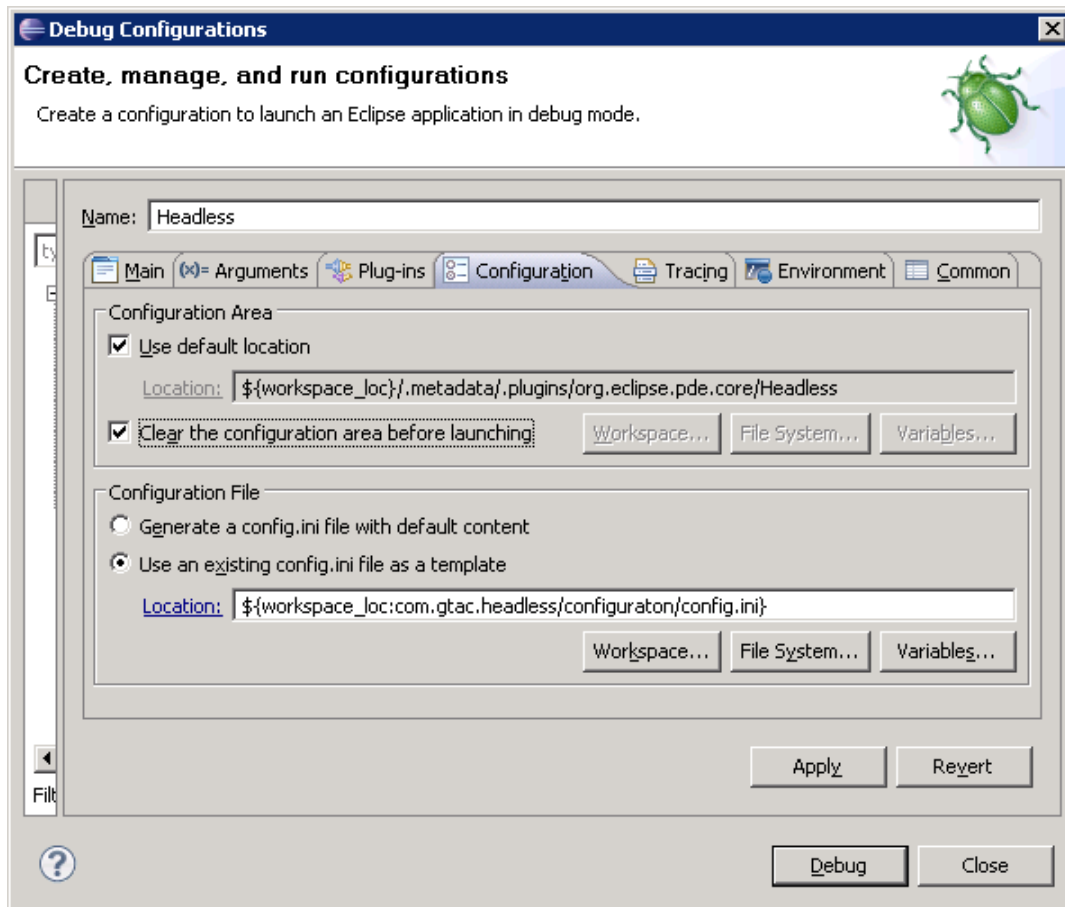


4. Check the **Clear** option, uncheck, **Ask for Confirmation** before clearing.
5. On the **Arguments tab**, for Program arguments, add: **-nosplash**.



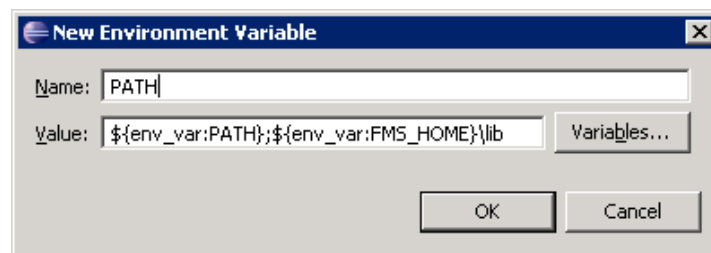
6. On the **Configuration tab**, select: Clear the configuration area before launching. (Important!)
7. Select: Use an existing config.ini file as template.
8. Click the **Workspace...** and select config.ini file.



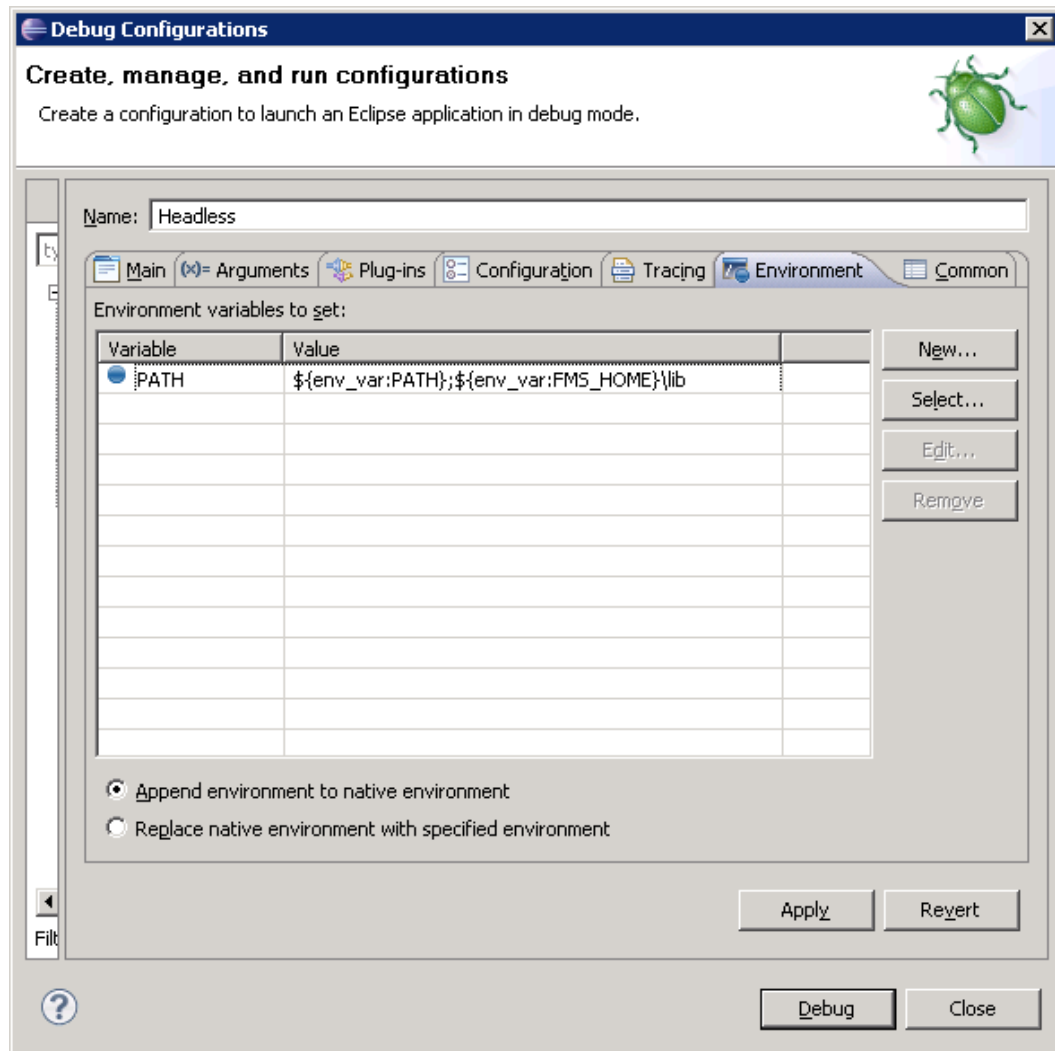


9. On the **Environment** tab add a New environment variable:

PATH=\${env_var:PATH};\${env_var:FMS_HOME}\lib



Note it is assumed FMS_HOME is set in your environment. If it is not, please set it.



10. Click **Apply**.

11. Click **Close**.

Debugging:

1. Start a Teamcenter TAO ImR window. This window is automatically started when you start the Teamcenter Rich Client session.
2. Open the Debug perspective. (Window, Open Perspective, Debug).
3. From the menu, select, **Run => Debug Configuration...**
4. Select **Headless** under Eclipse application.
5. Click **Debug**.
6. At some point the TAO window should show a Teamcenter Server Ready and trace information show up in the Console view.
7. Once the RAC session has fully initialized the debugger should break at the session line breakpoint.
8. Use the debugger to **Step**, **Step Into**, or **Run**.
9. Once completed you should be able to verify a new Item have been created in the users home folder using the Rich Client.

Patrick Hoonhout